

EDA

September 24, 2026

0.0.1 Packages

```
[3]: import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import plotly.figure_factory as ff
from IPython.display import display

pd.set_option('display.max_columns', None)
np.random.seed(1)
```

0.0.2 Data

```
[4]: # Load CSV files

# Test Data
testop = pd.read_csv('../data/2024-34-1/data/test_operational_readouts.csv')
testspec = pd.read_csv('../data/2024-34-1/data/test_specifications.csv')
# Train Data
trainop = pd.read_csv('../data/2024-34-1/data/train_operational_readouts.csv')
trainspec = pd.read_csv('../data/2024-34-1/data/train_specifications.csv')
traintte = pd.read_csv('../data/2024-34-1/data/train_tte.csv')
# Validation Data
vallab = pd.read_csv('../data/2024-34-1/data/validation_labels.csv')
valop = pd.read_csv('../data/2024-34-1/data/validation_operational_readouts.
↳csv')
valspec = pd.read_csv('../data/2024-34-1/data/validation_specifications.csv')

# Convert to DataFrames

# Test Data
df_testop = pd.DataFrame(data=testop)
df_testspec = pd.DataFrame(data=testspec)
# Train Data
df_trainop = pd.DataFrame(data=trainop)
df_trainspec = pd.DataFrame(data=trainspec)
df_traintte = pd.DataFrame(data=traintte)
```

```
# Validation Data
df_vallab = pd.DataFrame(data=vallab)
df_valop = pd.DataFrame(data=valop)
df_valspec = pd.DataFrame(data=valspec)
```

```
[3]: print(df_trainop.shape)
df_trainop.head()
```

```
(1122452, 107)
```

```
[3]:
```

	vehicle_id	time_step	171_0	666_0	427_0	837_0	167_0	\
0	0	11.2	167985.0	10787.0	7413813.0	2296.0	4110.0	
1	0	11.4	167985.0	10787.0	7413813.0	2296.0	4111.0	
2	0	19.6	331635.0	14525.0	13683604.0	2600.0	NaN	
3	0	20.2	354975.0	15015.0	14540449.0	2616.0	NaN	
4	0	21.0	365550.0	15295.0	14966985.0	2720.0	NaN	

	167_1	167_2	167_3	167_4	167_5	167_6	167_7	\
0	1296420.0	1628265.0	630345.0	1269525.0	4772940.0	2706706.0	222225.0	
1	1302855.0	1628265.0	630345.0	1269526.0	4772940.0	2706706.0	222225.0	
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

	167_8	167_9	309_0	272_0	272_1	272_2	272_3	\
0	6240.0	0.0	70.0	1435083.0	857662.0	384579.0	668642.0	
1	6240.0	0.0	70.0	1440661.0	857662.0	384579.0	668642.0	
2	NaN	NaN	70.0	1787736.0	1133132.0	598351.0	1167062.0	
3	NaN	NaN	70.0	1824409.0	1166074.0	634595.0	1233908.0	
4	NaN	NaN	70.0	1873614.0	1176071.0	639587.0	1266369.0	

	272_4	272_5	272_6	272_7	272_8	272_9	835_0	370_0	\
0	7239843.0	398490.0	3887.0	0.0	0.0	0.0	8036751.0	0.0	
1	7239843.0	398490.0	3887.0	0.0	0.0	0.0	8040811.0	0.0	
2	12314224.0	460240.0	3887.0	0.0	0.0	0.0	12777022.0	0.0	
3	13275730.0	466753.0	3887.0	0.0	0.0	0.0	13612083.0	0.0	
4	13758524.0	474047.0	3887.0	0.0	0.0	0.0	14041353.0	0.0	

	291_0	291_1	291_2	291_3	291_4	291_5	291_6	291_7	291_8	291_9	\
0	1227.0	555.0	463.0	925.0	468.0	225.0	535.0	516.0	492.0	729.0	
1	1230.0	558.0	463.0	925.0	469.0	226.0	535.0	516.0	493.0	729.0	
2	2136.0	954.0	850.0	1420.0	722.0	412.0	880.0	666.0	586.0	1143.0	
3	2218.0	1014.0	892.0	1471.0	749.0	425.0	901.0	702.0	589.0	1197.0	
4	2272.0	1038.0	910.0	1519.0	773.0	440.0	946.0	727.0	599.0	1233.0	

	291_10	158_0	158_1	158_2	158_3	158_4	158_5	\
0	66.0	97056.0	2690052.0	2945268.0	788437.0	687480.0	595164.0	
1	66.0	97056.0	2693100.0	2947368.0	788437.0	687480.0	595164.0	

2	162.0	181632.0	4249020.0	4630440.0	1539133.0	1421172.0	1039764.0
3	174.0	193728.0	4462548.0	4988028.0	1696022.0	1565484.0	1112544.0
4	178.0	203568.0	4621885.0	5107368.0	1758350.0	1633717.0	1159656.0

	158_6	158_7	158_8	158_9	100_0	459_0	459_1 \
0	491232.0	532932.0	809628.0	505693.0	858410.0	203.676778	111.911500
1	491232.0	532932.0	809628.0	505693.0	860571.0	204.256750	112.924250
2	749472.0	740724.0	995796.0	574045.0	1379191.0	321.671972	157.312500
3	789228.0	774588.0	1015104.0	576901.0	1428606.0	331.479028	162.731639
4	820368.0	792996.0	1030644.0	587234.0	1460547.0	339.563444	166.705583

	459_2	459_3	459_4	459_5	459_6	459_7 \
0	147.265389	200.479944	230.306278	277.722417	315.748806	372.164528
1	147.265389	201.479944	230.306278	277.722417	315.748806	372.164528
2	193.792833	263.577611	310.711861	366.149250	415.642472	484.391167
3	198.104472	269.712889	320.087333	377.478667	425.901361	495.749583
4	202.430194	275.669056	327.877944	385.460472	435.321167	506.110639

	459_8	459_9	459_10	459_11	459_12 \
0	864.246250	920.881111	637.901639	744.618944	880.866889
1	864.246250	920.881111	637.901639	745.618944	880.866889
2	1146.111611	1286.536333	900.062917	1123.232556	1449.545611
3	1173.882583	1323.460972	923.099361	1161.893139	1501.973944
4	1201.040833	1351.811444	944.133250	1192.740333	1544.355778

	459_13	459_14	459_15	459_16	459_17	459_18 \
0	1272.323972	1847.623667	940.785694	2.900083	0.208444	0.056417
1	1272.323972	1847.623667	940.785694	2.900083	0.208444	0.056417
2	2140.037472	5046.748278	1151.010139	3.320194	0.218806	0.056778
3	2208.782833	5587.856667	1160.593833	3.336417	0.218806	0.056778
4	2307.900278	5738.170167	1182.611833	3.367250	0.218806	0.056778

	459_19	397_0	397_1	397_2	397_3	397_4	397_5	397_6 \
0	0.058444	446956.0	411420.0	203024.0	26636.0	29156.0	7616.0	449537.0
1	0.058444	446964.0	411420.0	203027.0	26638.0	29157.0	7616.0	451193.0
2	0.058444	756665.0	647348.0	286811.0	30967.0	31213.0	7745.0	633790.0
3	0.058444	812577.0	686860.0	302955.0	31927.0	31488.0	7749.0	651902.0
4	0.058444	843929.0	710424.0	314292.0	32585.0	31896.0	7773.0	665154.0

	397_7	397_8	397_9	397_10	397_11	397_12	397_13	397_14 \
0	233352.0	139920.0	12648.0	2813.0	224.0	53161.0	178881.0	138250.0
1	233354.0	139920.0	12649.0	2813.0	224.0	53210.0	178883.0	138252.0
2	423395.0	271940.0	16190.0	3573.0	232.0	75038.0	352791.0	327992.0
3	478279.0	292109.0	16755.0	3753.0	232.0	77118.0	394083.0	359060.0
4	488271.0	300083.0	17265.0	3998.0	232.0	79222.0	411299.0	371612.0

	397_15	397_16	397_17	397_18	397_19	397_20	397_21	397_22 \
--	--------	--------	--------	--------	--------	--------	--------	----------

0	13328.0	3581.0	88.0	16361.0	131601.0	116541.0	13506.0	2856.0
1	13328.0	3582.0	88.0	16368.0	131601.0	116542.0	13507.0	2856.0
2	17325.0	4451.0	92.0	24028.0	234737.0	216619.0	17000.0	3476.0
3	17941.0	4573.0	92.0	25164.0	253706.0	232912.0	17583.0	3573.0
4	18614.0	4681.0	92.0	25736.0	268834.0	242477.0	18108.0	3641.0

	397_23	397_24	397_25	397_26	397_27	397_28	397_29	397_30 \
0	48.0	6337.0	105412.0	95728.0	15609.0	1984.0	8.0	784.0
1	48.0	6339.0	105413.0	95729.0	15610.0	1984.0	8.0	784.0
2	48.0	12055.0	167693.0	142900.0	19263.0	2441.0	12.0	1420.0
3	48.0	13199.0	176596.0	150565.0	19832.0	2522.0	12.0	1444.0
4	52.0	13400.0	183724.0	155913.0	20573.0	2562.0	12.0	1445.0

	397_31	397_32	397_33	397_34	397_35
0	150228.0	261904.0	93172.0	17874.0	452.0
1	150228.0	261905.0	93172.0	17874.0	452.0
2	204832.0	313485.0	106464.0	19306.0	452.0
3	211688.0	318901.0	107745.0	19406.0	453.0
4	213956.0	323997.0	109514.0	19535.0	454.0

```
[4]: print(df_trainspec.shape)
df_trainspec.head()
```

(23550, 9)

```
[4]:   vehicle_id Spec_0 Spec_1 Spec_2 Spec_3 Spec_4 Spec_5 Spec_6 Spec_7
0           0   Cat0   Cat0   Cat0   Cat0   Cat0   Cat0   Cat0   Cat0
1           2   Cat0   Cat1   Cat1   Cat0   Cat0   Cat0   Cat0   Cat1
2           3   Cat0   Cat1   Cat1   Cat1   Cat0   Cat0   Cat0   Cat1
3           4   Cat0   Cat0   Cat2   Cat1   Cat0   Cat0   Cat0   Cat1
4           5   Cat0   Cat2   Cat2   Cat0   Cat0   Cat0   Cat0   Cat1
```

```
[5]: print(df_traintte.shape)
df_traintte.head()
```

(23550, 3)

```
[5]:   vehicle_id length_of_study_time_step in_study_repair
0           0                        510.0              0
1           2                        281.8              0
2           3                        293.4              0
3           4                        210.0              0
4           5                        360.4              0
```

```
[5]: def add_temporal_aggregates(df_indexed, sensor_cols):
    """
    Adds expanding-window temporal aggregate features (last, mean, std,
    slope, delta) for every sensor column, computed per vehicle using
```

only that vehicle's readings up to and including each row -- never future readings, to avoid leakage into earlier-timestep labels.

Performance note: slope is computed via closed-form cumulative sums (a standard least-squares trick) instead of .expanding().apply(), which would call a Python function once per row per sensor -- far too slow at this dataset's scale (1.1M+ rows x 105 sensors) and the kind of thing that previously crashed the kernel on this data.

Args:

df_indexed (DataFrame): operational readouts, MUST already be sorted by vehicle_id then time_step (expanding logic assumes row order == chronological order within each vehicle).
sensor_cols (list of str): sensor column names to aggregate.

Returns:

df_indexed (DataFrame): same dataframe, with 5 new columns added per sensor (e.g. "397_0_last", "397_0_mean", "397_0_std", "397_0_slope", "397_0_delta").

```
"""
grouped_by_vehicle = df_indexed.groupby(level=0)

for sensor in sensor_cols:
    values = df_indexed[sensor]
    valid = values.notna()

    # last: just the raw reading itself, no computation needed
    df_indexed[f"{sensor}_last"] = values

    # mean / std: pandas' built-in expanding ops already skip
    # NaNs correctly, and are implemented in fast compiled code
    df_indexed[f"{sensor}_mean"] = grouped_by_vehicle[sensor].expanding().
↪mean().values
    df_indexed[f"{sensor}_std"] = grouped_by_vehicle[sensor].expanding().
↪std().values

    # delta: current value minus this vehicle's very first
    first_value = df_indexed.groupby(level=0)[sensor].transform("first")
    df_indexed[f"{sensor}_delta"] = values - first_value

    # slope: expanding least-squares slope of sensor value vs.
    time_step = df_indexed["time_step"]

    # zero out invalid (NaN-sensor) rows' contribution to the sums,
    # so missing readings are skipped rather than treated as 0
    x = time_step.where(valid, 0)
    y = values.where(valid, 0)
```

```

xy = (time_step * values).where(valid, 0)
xx = (time_step ** 2).where(valid, 0)
n = valid.astype(int) # counts only valid readings toward n

# cumulative sums per vehicle, using only valid readings so far
cum_n = grouped_by_vehicle[n.name].cumsum() if hasattr(n, "name") else
↳ n.groupby(df_indexed.index).cumsum()

tmp = pd.DataFrame({
    "n": n, "x": x, "y": y, "xy": xy, "xx": xx
}, index=df_indexed.index)
cum = tmp.groupby(level=0).cumsum()

# standard least-squares slope formula, applied elementwise:
# slope = (n*sum(xy) - sum(x)*sum(y)) / (n*sum(xx) - sum(x)^2)
numerator = cum["n"] * cum["xy"] - cum["x"] * cum["y"]
denominator = cum["n"] * cum["xx"] - cum["x"] ** 2

slope = numerator / denominator

slope[(cum["n"] < 2) | (denominator == 0)] = np.nan

df_indexed[f"{sensor}_slope"] = slope

df_indexedtrainop_idx = df_trainop.set_index("vehicle_id").sort_index()

return df_indexedtrainop_idx

trainop_idx = df_trainop.set_index("vehicle_id").sort_index()
testop_idx = df_testop.set_index("vehicle_id").sort_index()
valop_idx = df_valop.set_index("vehicle_id").sort_index()

def getsetname(df_indexed):
    """ Provide the name of the set type for string interpolation.
    Args:
        df_indexed (): Pandas DataFrame index to the corresponding dataset.
    Returns:
        (String): The name of the dataset as a string.
    """

    if df_indexed is trainop_idx:
        return "training set"
    elif df_indexed is testop_idx:
        return "test set"
    elif df_indexed is valop_idx:

```

```

        return "validation set"

def sensor_readouts(df_indexed, graph_type, vehicle_id=None):
    """Produce a multi-variate time series chart that displays sensor readouts
    over time...
    until the vehicle is outside of its monitoring window.
    Args:
        vehicle_id (Integer): Target for Pandas DataFrame to extract a
        vehicle's unique sensor data.
    """
    Set = getsetname(df_indexed)

    if vehicle_id is not None and vehicle_id not in df_indexed.index:
        print(
            f"Invalid Vehicle ID: Vehicle ID {vehicle_id} is either out of
            range or not included in the {set} set."
        )
        return None

    full_data = df_indexed.reset_index()

    if vehicle_id is not None:
        df = df_indexed.loc[[vehicle_id]].reset_index()
        data = df.iloc[:, 1:]

    sensors = [c for c in full_data.iloc[:, 2:].columns]
    cols = [c for c in full_data.iloc[:, 1:].columns]

    for c in cols:
        if full_data[c].isna().all():
            continue
        col_min, col_max = np.nanmin(full_data[c]), np.nanmax(full_data[c])
        if col_max == col_min:
            continue
        if vehicle_id is not None:
            data[c] = (data[c] - col_min) / (col_max - col_min)
        else:
            full_data[c] = (full_data[c] - col_min) / (col_max - col_min)

    if vehicle_id is not None:
        long_df_single = data.melt(
            id_vars="time_step",
            value_vars=sensors,
            var_name="sensor",
            value_name="value",

```

```

)

fig0 = px.line(
    long_df_single,
    x="time_step",
    y="value",
    color="sensor",
    title=f"{Set.title()} Sensor Readouts | Vehicle ID {vehicle_id}",
    render_mode="webgl",
    template="plotly_dark"
)
fig0.update_layout(height=600)

fig1 = px.imshow(
    data[sensors].corr(),
    x=sensors,
    y=sensors,
    labels={"x": "sensor", "y": "sensor", "color": "correlation"},
    template="plotly_dark",
    title=f"{Set.title()} Sensor Readouts | Vehicle ID {vehicle_id}",
)
fig1.update_layout(height=800)

else:
    long_df_all = full_data.melt(
        id_vars="time_step",
        value_vars=sensors,
        var_name="sensor",
        value_name="value",
    )

    agg = long_df_all.groupby(["time_step", "sensor"], as_index=False)[
        "value"
    ].mean()
    wide = agg.pivot(index="time_step", columns="sensor", values="value")

    fig2 = px.line(
        agg,
        x="time_step",
        y="value",
        color="sensor",
        title=f"{Set.title()} Sensor Readouts | All Vehicles",
        render_mode="webgl",
        template="plotly_dark"
    )
    fig2.update_layout(height=600)

```

```

fig3 = px.imshow(
    wide.corr(),
    x=sensors,
    y=sensors,
    labels={"x": "sensor", "y": "sensor", "color": "correlation"},
    template="plotly_dark",
    title=f"{Set.title()} Sensor Readouts Mean Correlation | All_
↳Vehicles",
)
fig3.update_layout(height=800)

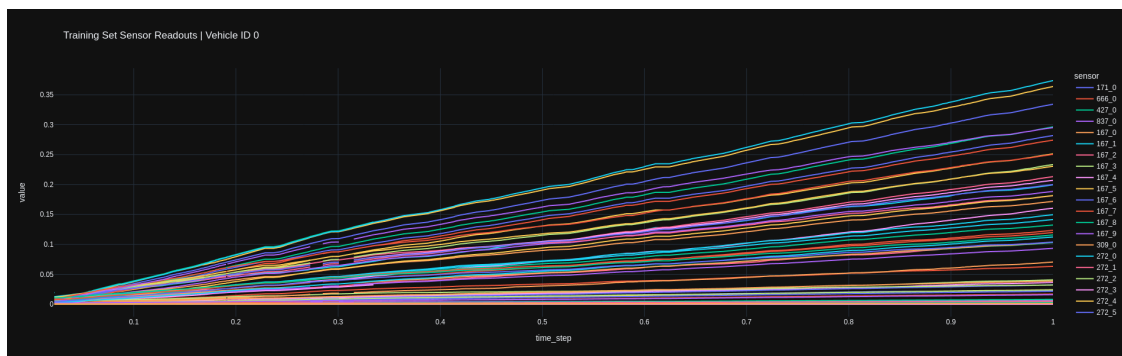
if graph_type == "line" and vehicle_id is not None:
    fig0.show()
elif graph_type == "correlation" and vehicle_id is not None:
    fig1.show()
if graph_type == "line" and vehicle_id is None:
    fig2.show()
elif graph_type == "correlation" and vehicle_id is None:
    fig3.show()

```

```

[7]: # Edit the vehicle ID and specify the set index to plot its individual sensor_
↳data overtime.
sensor_readouts(
    trainop_idx, # trainop_idx, testop_idx or valop_idx
    'line',
    0
)

```

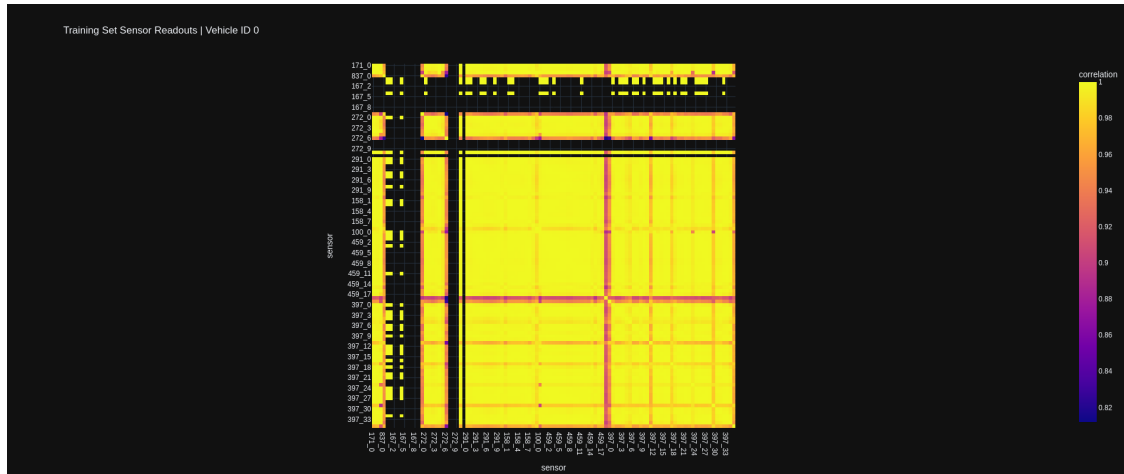


```

[8]: # Edit the vehicle ID and specify the set index to compare sensor-to-sensor_
↳correlation per vehicle.
sensor_readouts(
    trainop_idx, # trainop_idx, testop_idx or valop_idx
    'correlation',
    0 # Number between 0 and x
)

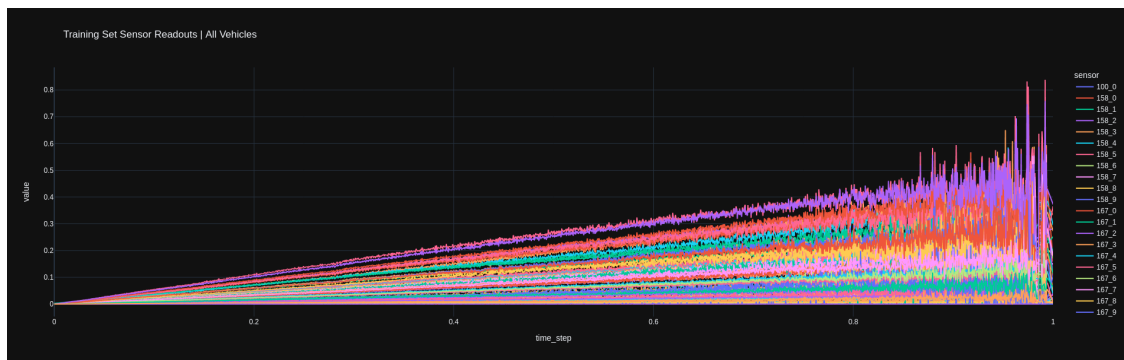
```

)



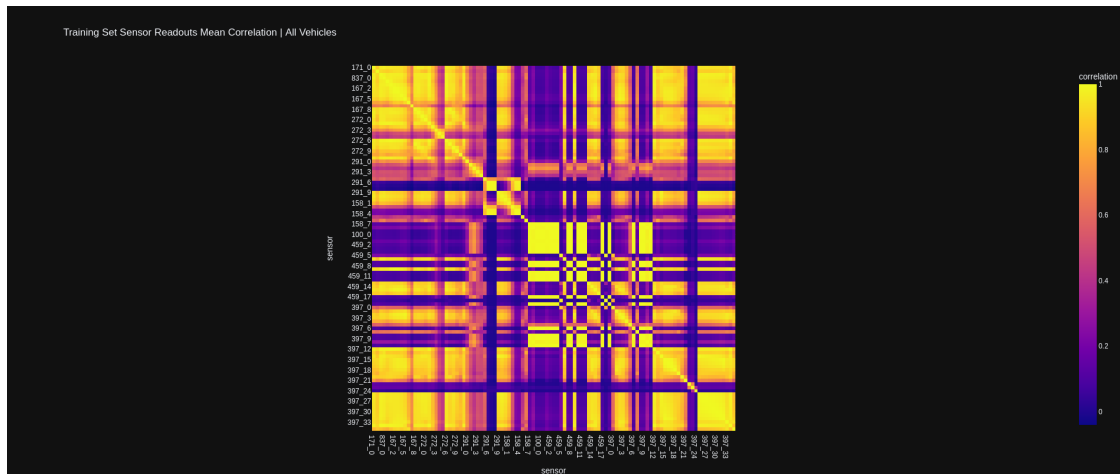
[6]: # Edit the set index to see how mean sensor readouts change over timesteps set_
→wide.

```
sensor_readouts(
    trainop_idx, # trainop_idx, testop_idx or valop_idx
    'line',
)
```



[7]: # Edit the set index to see how mean sensor readouts over time correlate sensor_
→to sensor set wide.

```
sensor_readouts(
    trainop_idx, # trainop_idx, testop_idx or valop_idx
    'correlation', # line or correlation
)
```



Findings:

- no sensor readout pairs display a negative correlation
- sensor readouts display consistent NaN values for many vehicles that do not correlate with other sensors
- sensor readouts trend toward linear growth with minimal volatility for individual vehicles
- mean sensor readouts spanning the entire set trend toward linear growth with volatility that increases as time steps and sensor values grow

```
[8]: def spec_counts_row(dfset):
    """ Finds the unique spec rows in a dataset and describes the distribution
    of specifications.
    Args:
        dfset (Pandas: DataFrame):
    Returns:
        labels (list): list of placeholder names for row partitions.
        values (list): list of counts for vehicles of the same specification.
    """
    specs = dfset.iloc[:, 1:] # Drop the vehicle ID column
    specs.drop_duplicates(inplace=True)
    spec_cols = specs.columns.tolist()
    counts = dfset.groupby(spec_cols).size()
    values = [counts.get(tuple(row), 0) for row in specs.
    itertuples(index=False)]
    labels = [f"spec_{i}" for i in range(len(specs))]
    return labels, values
```

```
[9]: labels, values = spec_counts_row(df_trainspec)

counts = pd.Series(values, index=labels).sort_values(ascending=False)
print(counts.describe())
```

```
print((counts.cumsum() / counts.sum()).head(20))
```

```
count      3607.000000
mean        6.528971
std         59.296543
min          1.000000
25%          1.000000
50%          1.000000
75%          3.000000
max         2378.000000
dtype: float64
fspec_45      0.100977
fspec_1318    0.176136
fspec_5       0.223100
fspec_10      0.251805
fspec_1       0.279958
fspec_2       0.305902
fspec_8       0.326709
fspec_1381    0.346752
fspec_1322    0.364416
fspec_111     0.381953
fspec_1450    0.397877
fspec_1463    0.413588
fspec_1514    0.423652
fspec_20      0.430701
fspec_3206    0.437537
fspec_17      0.443227
fspec_12      0.448705
fspec_13      0.454013
fspec_67      0.459151
fspec_42      0.464076
dtype: float64
```

findings:

- Parsing unique specification combinations by rows leads to few unique partitions that can not be used to describe any pattern in the data at scale alone. (3000 > unique combinations)

```
[10]: def spec_counts_col(dfset):
        """ Finds the unique values within each spec column and describes the
            distribution of specifications, per column.
        Args:
            dfset (Pandas: DataFrame):
        Returns:
            results (dict): maps each spec column name to a (labels, values) tuple,
                where labels are placeholder names for that column's unique values
                and values are the counts of vehicles matching each one.
        """
```

```

specs = dfset.iloc[:, 1:] # Drop the vehicle ID column
spec_cols = specs.columns.tolist()

results = {}
for col in spec_cols:
    counts = dfset[col].value_counts()
    labels = [f"{col}_{i}" for i in range(len(counts))]
    values = counts.tolist()
    results[col] = (labels, values)

return results

```

```

[11]: for col, (labels, values) in spec_counts_col(df_trainspec).items():
        counts = pd.Series(values, index=labels).sort_values(ascending=False)
        print(f"--- {col} ---")
        print(counts.describe())
        print((counts.cumsum() / counts.sum()).head(20))

```

```

--- Spec_0 ---
count      3.0000
mean      7850.0000
std      10074.5587
min       38.0000
25%      2164.5000
50%      4291.0000
75%     11756.0000
max     19221.0000
dtype: float64
Spec_0_0    0.816178
Spec_0_1    0.998386
Spec_0_2    1.000000
dtype: float64
--- Spec_1 ---
count     29.000000
mean      812.068966
std      2446.389044
min        1.000000
25%       38.000000
50%      278.000000
75%      676.000000
max     13362.000000
dtype: float64
Spec_1_0    0.567389
Spec_1_1    0.642038
Spec_1_2    0.681741
Spec_1_3    0.718259
Spec_1_4    0.752994
Spec_1_5    0.784798

```

```

Spec_1_6      0.815202
Spec_1_7      0.843907
Spec_1_8      0.866879
Spec_1_9      0.887856
Spec_1_10     0.904841
Spec_1_11     0.921146
Spec_1_12     0.934565
Spec_1_13     0.947771
Spec_1_14     0.959575
Spec_1_15     0.968917
Spec_1_16     0.975754
Spec_1_17     0.982590
Spec_1_18     0.988025
Spec_1_19     0.992611
dtype: float64
--- Spec_2 ---
count         21.000000
mean          1121.428571
std           3152.582363
min            4.000000
25%           17.000000
50%           88.000000
75%          545.000000
max          14352.000000
dtype: float64
Spec_2_0      0.609427
Spec_2_1      0.747643
Spec_2_2      0.859066
Spec_2_3      0.893546
Spec_2_4      0.925605
Spec_2_5      0.948747
Spec_2_6      0.960127
Spec_2_7      0.968875
Spec_2_8      0.976603
Spec_2_9      0.983524
Spec_2_10     0.987261
Spec_2_11     0.990743
Spec_2_12     0.993546
Spec_2_13     0.996263
Spec_2_14     0.997495
Spec_2_15     0.998217
Spec_2_16     0.998896
Spec_2_17     0.999321
Spec_2_18     0.999660
Spec_2_19     0.999830
dtype: float64
--- Spec_3 ---
count          4.000000

```

```

mean      5887.500000
std       5882.168336
min       1871.000000
25%       2086.250000
50%       3620.500000
75%       7421.750000
max       14438.000000
dtype: float64
Spec_3_0   0.613079
Spec_3_1   0.828917
Spec_3_2   0.920552
Spec_3_3   1.000000
dtype: float64
--- Spec_4 ---
count      2.000000
mean      11775.000000
std       15697.770542
min        675.000000
25%       6225.000000
50%      11775.000000
75%      17325.000000
max      22875.000000
dtype: float64
Spec_4_0   0.971338
Spec_4_1   1.000000
dtype: float64
--- Spec_5 ---
count      5.000000
mean      4710.000000
std       8077.055652
min        62.000000
25%       740.000000
50%      1016.000000
75%      2676.000000
max     19056.000000
dtype: float64
Spec_5_0   0.809172
Spec_5_1   0.922803
Spec_5_2   0.965945
Spec_5_3   0.997367
Spec_5_4   1.000000
dtype: float64
--- Spec_6 ---
count     17.000000
mean     1385.294118
std     2923.340058
min        1.000000
25%        9.000000

```

```

50%          24.000000
75%          986.000000
max         10163.000000
dtype: float64
Spec_6_0     0.431550
Spec_6_1     0.744968
Spec_6_2     0.863397
Spec_6_3     0.933248
Spec_6_4     0.975117
Spec_6_5     0.989597
Spec_6_6     0.995244
Spec_6_8     0.996263
Spec_6_7     0.997282
Spec_6_9     0.997962
Spec_6_10    0.998556
Spec_6_11    0.999108
Spec_6_12    0.999490
Spec_6_13    0.999745
Spec_6_14    0.999915
Spec_6_15    0.999958
Spec_6_16    1.000000
dtype: float64
--- Spec_7 ---
count         9.000000
mean        2616.666667
std         2486.098047
min          265.000000
25%          892.000000
50%         2043.000000
75%         4284.000000
max         7714.000000
dtype: float64
Spec_7_0     0.327558
Spec_7_1     0.525605
Spec_7_2     0.707516
Spec_7_3     0.803185
Spec_7_4     0.889936
Spec_7_5     0.937155
Spec_7_6     0.975032
Spec_7_7     0.988747
Spec_7_8     1.000000
dtype: float64

```

Findings:

- Parsing by unique columns causes far fewer sparse pairings than rows

[]: